

Triage: An Adaptive Parallel Window Decoding Scheduler for Real-time Fault-Tolerant Quantum Computation

1st Jiahan Chen*

The Hong Kong University of
Science and Technology (Guangzhou)
Guangzhou, China

2nd Chenghong Zhu*

The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China

3rd Ge Bai†

The Hong Kong University of
Science and Technology (Guangzhou)
Guangzhou, China
gebai@hkust-gz.edu.cn

4th Xin Wang†

The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China
felixxinwang@hkust-gz.edu.cn

Abstract—Fault-tolerant quantum computation (FTQC) critically depends on real-time classical decoding, which is rapidly emerging as a system bottleneck. As quantum systems scale, decoding latency and throughput limitations lead to exponential syndrome backlogs and logical operation stalls. While hardware accelerators and parallel windowing offer pathways to speed up decoding, dynamically deploying a finite pool of decoders across a vast quantum error correction architecture remains an unresolved resource allocation problem.

To address this, we formulate FTQC decoding as a constrained dynamic scheduling problem by utilizing a spatio-temporal framework based on *slices*. We propose Triage, a dual-mode architecture that mitigates operation stalls by adaptively combining a cost-efficient heuristic scheduler with a priority-aware emergency mode to rapidly resolve the causal cone of critical operations. Our evaluation shows that Triage maintains low algorithm stalls and logical error rates even under scarce classical resource constraints. Across various benchmarks, Triage achieves an average logical error rate reduction of 52.6% compared to standard temporal parallelism, enabling an efficient classical control plane for scalable FTQC architectures.

Index Terms—Fault-tolerant quantum computing, quantum error correction, real-time decoding, parallel window decoding.

I. INTRODUCTION

Quantum computers hold the potential to efficiently solve certain problems that are intractable for the best known classical algorithms [1]–[3]. However, current quantum hardware is highly error-prone [4], [5], requiring quantum error

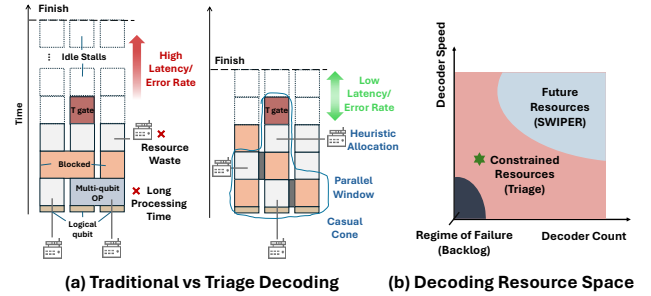


Fig. 1. Navigating the FTQC Decoding Bottleneck. (a) Traditional decoding leads to large idle stalls, while Triage employs spatio-temporal windows and prioritizes the causal cone to effectively reduce the latency. (b) Triage achieves better performance in the near-term, resource-constrained landscape.

correction (QEC) to enable fault-tolerant quantum computation (FTQC) [6]. Encouragingly, recent experimental progress across various platforms and QEC codes [7]–[9] is rapidly validating this approach [10]–[14]. As these advances move FTQC from theory toward viability, the focus shifts from theoretical feasibility to the architectural challenges of implementation.

Towards a large scale FTQC, a significant architectural bottleneck arises from the classic control layer. At the heart of this layer is the *decoder*, whose function is to continuously process a massive stream of classical syndrome data from the quantum processor, and infer the most likely errors. Crucially, decoding must operate in *real-time*. This means the overall throughput of decoding must, on average, exceed the rate of syndrome generation. Otherwise, the system will accumulate an exponential backlog [15] of unprocessed syndromes, which will eventually overwhelm the computational resources.

To address this challenge, one line of research focuses on optimizing the *latency* of a single decoding operation. On

*Co-first authors

†Co-corresponding authors

This work has been partially supported by the National Key R&D Program of China (Grant No. 2024YFB4504004), the National Natural Science Foundation of China (Grant No. 12447107), the Guangdong Provincial Quantum Science Strategic Initiative (Grant Nos. GDZX2403008 and GDZX2503001), and the Guangdong Provincial Key Lab of Integrated Communication, Sensing and Computation for Ubiquitous Internet of Things (Grant No. 2023B1212010007).

the software front, significant effort has gone into developing high-accuracy decoding algorithms with low computational complexity [16]–[19]. In parallel, hardware accelerators using FPGAs have demonstrated single-decode latencies below the demands of superconducting qubits for certain code distances [20]–[23]. However, these hardware demonstrations have been largely confined to memory experiments rather than integrated into logical computations. Meanwhile, the protocol that processes the syndrome stream in a *serial* fashion has been shown to be unscalable [24]. Therefore, beyond optimizations for latency, designing superior decoding protocols that enhance overall *throughput* is essential.

A second direction aims to improve decoding throughput via *parallelism*. Temporal parallelism, for instance, partitions the syndrome stream into time blocks, allowing concurrent processing of the non-adjacent blocks [24]–[26]. For multi-qubit logical operations such as lattice surgery, spatial parallelism can also be employed by partitioning the syndromes from the involved regions [27]. In principle, a spatio-temporal parallel approach would allow the system to scale its total throughput simply by adding more decoder units. Nevertheless, while temporal or spatial windowing techniques are known, a systematic scheduling framework integrating resource-aware temporal and spatial parallelism has not yet been demonstrated.

How to deploy a finite pool of decoders onto a FTQC application? First, there is an *asymmetry* between classical resources and logical qubits. Depending on hardware limitations or error targets, an application may require a few high-speed decoders for small codes, or a large pool of decoders collaborating in parallel for large codes. In a realistic large-scale architecture, this necessitates an M -for- N shared resource model where $M < N$ decoders must be dynamically allocated. At any moment, determining *which* decoder to assign to *which* logical patch is a resource allocation problem [28].

This scheduling problem is further complicated by the operational logic of FTQC. The *Pauli frame* [29] stores the decoder’s inference of accumulated errors. When the computation encounters a non-Clifford gate, the decoder must update all relevant Pauli frames, a process we term *synchronization*. A synchronization failure forces the logical operation to stall. During this idle period, qubits undergo additional error correction rounds, directly increasing the logical error rate (LER). To maximize fidelity via idle-reduction, decoding tasks relevant to the critical non-Clifford operation must be given higher priority. Combining this urgency with the spatio-temporal dependency constraints from parallel decoding, the problem is transformed into a dynamic constrained scheduling problem. As illustrated in Figure 1, traditional decoding approaches fail to navigate these dependencies efficiently, leading to severe resource waste and long idle stalls. Furthermore, as the hardware design space spans diverse decoder speeds and counts, a robust scheduling strategy becomes critical, especially in near-term, resource-constrained environments where naive policies quickly fall into an unrecoverable regime of failure.

In this paper, we systematically address this challenge. First, we utilize a parallel spatio-temporal decoding framework using

slice (a $d \times d$ patch over d rounds) as the basic scheduling unit. By modeling the lifecycle of each slice and identifying the *causal cone* of critical operations, we formulate the FTQC decoder scheduling problem. Second, to optimize performance under constrained resources, we propose Triage, a dual-mode scheduling architecture which combines a fast heuristic-based steady mode with a robust look-ahead emergency mode. Triage significantly reduces the logical operation stalls, leading to an average LER reduction of 52.6% compared to the standard temporal-parallel scheduling strategy.

In summary, we make the following contributions:

- We introduce an abstraction of the decoder scheduling problem based on a constraint graph of *slices*. This framework is hardware-agnostic and applicable to diverse quantum platforms utilizing surface codes.
- We propose the Triage scheduler, a dual-mode system that minimizes logical operation stalls by dynamically invoking an emergency mode to rapidly resolve the causal cone of prerequisite decodes.
- We demonstrate that by effectively scheduling parallel windows, it is possible to overcome the latency limitations of individual decoders, enabling FTQC even in the challenging slow-decoder regime ($\tau_{\text{decode}} > \tau_{\text{syndrome}}$).
- We quantify the impact of real-time scheduling on system-level fidelity, presenting a simulation framework that captures the interaction between syndrome generation and decoding.

II. BACKGROUND

A. Preliminary

1) Quantum Computing and Quantum Error Correction:

The fundamental unit of quantum computing is the qubit. A qubit inhabits a 2-D Hilbert space with computational basis states $|0\rangle$ and $|1\rangle$, and an arbitrary pure state can be written as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$. Realistic qubits are noisy and error-prone. For example, a bit-flip error maps $|\psi\rangle$ to $\alpha|1\rangle + \beta|0\rangle$, and a phase-flip error maps $\alpha|0\rangle + \beta|1\rangle$ to $\alpha|0\rangle - \beta|1\rangle$. Quantum error correction (QEC) codes are necessary to preserve quantum information against such errors.

Stabilizer codes form a broad family of QEC codes that includes many of the most widely used constructions. A stabilizer code is specified by a set of commuting stabilizer generators $S_1, \dots, S_m$, each of which is a Pauli operator acting as a tensor product of Pauli strings on the physical qubits. During syndrome extraction, dedicated ancilla qubits interact with the data qubits to measure the parity of each stabilizer without collapsing the encoded state.

2) *Surface Code and Lattice Surgery*: The surface code [7] has emerged as a leading candidate for building practical fault-tolerant quantum computers due to its high error threshold and hardware compatibility. Fig. 2(a) shows a rotated surface code of distance $d = 3$. The corresponding syndrome extraction circuit is shown in Fig. 2(b)(c): circles represent data qubits, and each data qubit couples to adjacent X- and Z-type ancilla

qubits. The syndrome extraction is repeated over multiple rounds to collect measurement outcomes.

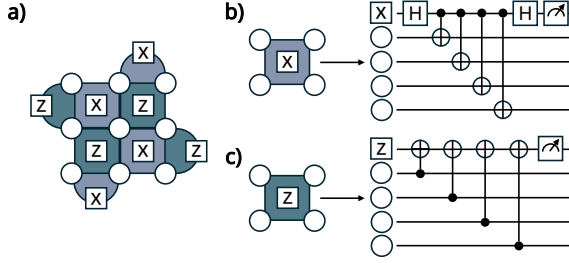


Fig. 2. Example rotated surface code of distance $d = 3$. a) The code is defined by a set of X- and Z-type stabilizer checks used for syndrome extraction. b) and c) Syndrome extraction circuits for the X and Z stabilizers, respectively.

Lattice surgery [30] is a leading approach for implementing logical operations in surface code architectures. It works by measuring joint stabilizers along the boundaries of adjacent code patches, temporarily merging and then splitting patches [30]–[32] to enact gate primitives. In contrast, non-Clifford operations such as the T gate, are typically supplied via magic state distillation or cultivation [33]–[37]. In this distillation process, multiple noisy magic states are converted into fewer, higher fidelity states that are suitable for fault-tolerant state injection.

Following the best practice [38], we represent each encoded surface-code qubit as a tile, as shown in Fig. 3(a). Building on this abstraction, Fig. 3(b–d) illustrates the key lattice surgery operations on tiles: patch movement, patch rotation, and multi-patch parity measurement. Execution of logical circuits typically follows the Pauli-Based Computation (PBC) paradigm, which systematically translates the universal Clifford+T circuits into a sequence of Pauli rotations. These rotations are then realized via the requisite lattice-surgery operations. We refer to [38] for a detailed introduction.

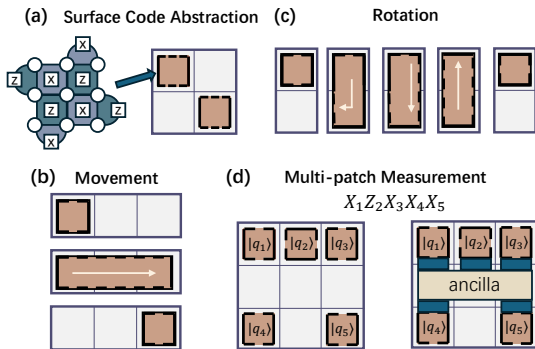


Fig. 3. (a) Abstract view of the surface code as a patch. (b–d) Summary of the logical operations that can be performed.

3) *The Pauli Frame and T-Gate Synchronization*: The classical processing requirements for FTQC are fundamentally dictated by the Pauli frame [29], [39] and its interaction with non-Clifford gates. The Pauli frame is a classical data structure

that efficiently tracks the accumulation of Pauli errors on data qubits. This is a direct consequence of the definition of the Clifford group $\mathcal{C}_n$. If an operation $C \in \mathcal{C}_n$ is a Clifford gate, then for any Pauli operator $P \in \mathcal{P}_n$, the transformation results in another Pauli operator $P' = CPC^\dagger \in \mathcal{P}_n$. This property allows for efficient classical tracking: if an accumulated error $E \in \mathcal{P}_n$ exists on the state $|\psi\rangle$, applying a Clifford circuit C transforms the state to $C(E|\psi) = (CEC^\dagger)(C|\psi) = E'(C|\psi)$. The new error E' is also a Pauli operator and can be easily computed classically, allowing the frame to be updated without physical correction.

This convenience ends, however, with the introduction of non-Clifford gates, such as the T-gate, which are essential for universal quantum computation [33]. The T-gate breaks the classical tracking mechanism, as the transformed error can no longer be represented in the Pauli frame; for instance, $TXT^\dagger \notin \mathcal{P}_n$. As illustrated in Figure 4, a T-gate is typically implemented via preparing a high-fidelity magic state [34], [37], [40] and realizing a gate teleportation, which concludes with a measurement and a classically-controlled Pauli correction (an S-gate). Crucially, this final correction cannot be commuted through the T-gate and absorbed into the Pauli frame. Before this correction can be applied, the accumulated error on the logical qubit, E_{acc} , must be physically corrected by applying $E_{acc}^\dagger$. Only after the state is restored from $E_{acc}|\psi\rangle$ to $|\psi\rangle$ can the teleportation proceed correctly.

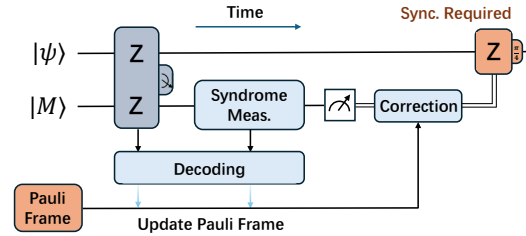


Fig. 4. T-gate implementation via gate teleportation. The classically-controlled S-gate correction forces a decoder synchronization by physically correcting the Pauli frame.

The central insight is the dichotomy in decoding requirements for FTQC. While the Pauli frame permits a relaxed, asynchronous approach to error correction, the presence of non-Clifford gates creates absolute synchronization points. They transform decoding into a priority scheduling problem, where synchronization failures lead to computational stalls and increased logical error rates.

4) *Window Decoding*: To manage the continuous stream of syndrome data in FTQC, decoders operate on discrete chunks of information known as windows. The traditional approach is *serial sliding window* decoding [41], where the temporal syndrome data is partitioned into fixed-size windows that are processed sequentially. However, this approach faces a scalability bottleneck. Let the time to generate the data for one window be τ_{gen} and the time for a single decoder to process it be τ_{dec} . To prevent an exponential backlog of unprocessed syndromes, the system must satisfy the condition

$\tau_{dec} < \tau_{gen}$ [15]. Assuming a decoder whose latency scales linearly with the number of qubits, N , i.e., $\tau_{dec} \propto N$, this constraint can be rewritten as:

$$N < \frac{\tau_{round}}{k} \quad (1)$$

where τ_{round} is the duration of a single syndrome measurement cycle and k is a constant. This inequality reveals that for any decoder hardware, there exists an upper bound on the code distance that can be supported in real-time, rendering the approach unscalable for large QEC codes.

The introduction of *parallel window decoding* offered a solution. The key insight is that temporally disjoint windows are causally independent and can thus be decoded concurrently, as illustrated in Figure 5. In the time dimension, this allows for a checkerboard pattern of decoding, where all *even* windows can be processed in parallel, followed by all *odd* windows [24], [25]. This concept extends naturally to the spatial dimension, where operations on different logical qubits can also be partitioned and processed in parallel [27]. Further refinements, such as speculative decoding [26], aim to minimize the overhead at window boundaries. The parallel window approaches require that the decoding volume for a given window is expanded to include a buffer region containing syndrome data from its neighbors, with the window buffer size determining the extent of this look-ahead information. Then, the earlier decoded window creates artificial syndromes on its boundary with its neighbors. The insight of parallel window decoding is: with a sufficient number of parallel decoders, the system's overall throughput can be maintained even if individual decoders are slow ($\tau_{dec} \geq \tau_{gen}$).

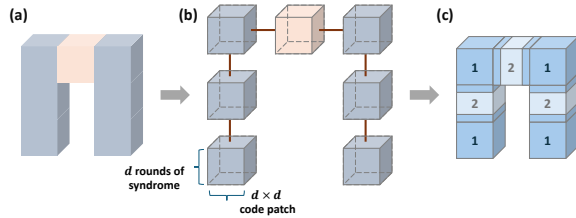


Fig. 5. Spatio-temporal partitioning of a lattice surgery operation. (a) The monolithic operation volume. (b) It is decomposed into a graph of causally-constrained slices, where red edges represent mutual exclusion constraints. (c) The graph is 2-colored, which partitions all slices into two independent sets, and each set can be decoded in parallel.

B. Motivation

1) *The Decoder Resource Dilemma*: Consider a surface code computation with a 2-D array of logical qubits, as shown in Figure 3. A real-time decoding system must continuously process the syndrome streams from all active qubit patches. How should we allocate resources in this classical system?

On one extreme, a *one-to-one mapping* dedicates a physical decoder to each logical qubit. While maximizing parallelism, this approach is architecturally infeasible for large systems due to its prohibitive cost and resource underutilization. On

the other extreme, a *one-for-all approach* uses a constant number of decoders to service the entire machine. This is also unscalable, as the latency requirement for each decoder would scale inversely with the number of logical qubits ($O(1/N_{lq})$), an impossible demand for any non-trivial algorithm.

The only viable path is a shared-resource model: an *M-for-N scheduler* that manages a pool of M physical decoders to service the tasks from N logical qubits, transforming the issue into a scheduling problem. The scheduler should make online decisions to prioritize tasks and maximize throughput, especially in two critical scenarios: the resource-constrained regime where fast decoders are scarce ($M \leq N, \tau_{dec} < \tau_{gen}$), and the computationally-constrained regime where decoders may be individually slow ($M > N, \tau_{dec} \geq \tau_{gen}$).

Recent research has begun exploring this scheduling problem [28]. Existing models, based on a logical qubit level abstraction, offer a valuable first step but fail to capture the fine-grained complexity inherent in large-scale quantum algorithms. By shifting the scheduling abstraction down to the level of individual spatio-temporal windows, we can unlock two powerful dimensions of optimization.

First, a fine-grained scheduler can effectively manage the strong *spatial correlations* created by operations like lattice surgery. Because lattice surgery temporarily merges adjacent logical patches and measures joint stabilizers along their boundaries, errors become spatially correlated across the merged region. Instead of forcing a single decoder to serialize the decoding of this massive, combined volume, a slice-aware scheduler can partition it into smaller, spatially parallelizable windowed tasks, simultaneously ensuring correctness and efficiency. Second, this approach can also fully leverage *spatio-temporal parallelism* during non-Clifford operations. A slice-aware scheduler can dispatch multiple decoders to collaboratively resolve emergent windows, reducing synchronization latency. Furthermore, this decomposition is inherently more efficient, as it avoids the super-linear complexity penalty associated with decoding large, monolithic data blocks [16].

2) *The Scalability Crisis of Parallel Decoding*: Parallel window decoding [24]–[26] have addressed the *computational scalability* problem of sliding window decoding. However, this arises an architectural question: what is the practical upper bound on the number of required decoders M ?

We define the complete set of dependencies for a critical operation as its *causal cone*: the transitive closure of all undecoded historical slices belonging to any logical qubit that has become correlated with the target through a chain of multi-qubit operations, as illustrated in Figure 6. In the worst case, if the algorithm is highly entangled and the intervals between non-Clifford gates are long, this causal cone can grow to a large spatio-temporal volume. A brute-force parallel approach to resolve such a backlog at the last minute would require a number of decoders proportional to the backlog's size.

This reveals the *resource scalability* as another potential crisis. The demand for decoders is highly non-uniform, with massive spikes preceding critical gates, making a static, worst-case provisioning of decoders architecturally infeasible. This

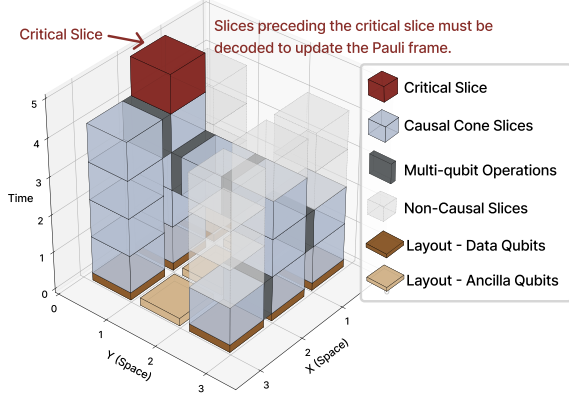


Fig. 6. An illustration of the causal cone corresponds to one critical slice, which is the Clifford correction after the T-gate teleportation in Figure 4.

insight is our central motivation: a dynamic online scheduler is essential to manage a finite decoder pool, handling the average workload efficiently while mobilizing maximum parallelism only when necessary to meet critical deadlines.

III. TRIAGE SCHEDULER

We first define the scheduler system model and then present our Triage scheduling algorithm.

A. System Model and Problem Formulation

The decoder scheduler serves as middleware within the FTQC classical control stack. As illustrated in Figure 7, its architecture is divided into offline and online phases.

1) *Offline Phase: Compile-Time Analysis:* Due to the superlinear complexity of the decoder, processing a large block of syndrome data is less efficient than decoding its components individually. Building upon the concepts of parallel window decoding, we leverage fine-grained spatio-temporal partitioning to maximize the degree of parallelism. We therefore adopt the *slice* as the atomic scheduling unit of our framework.

A slice $S(t, p)$, represents the syndrome data generated from a single square logical patch at position p during a d -rounds syndrome measurement cycle t . The scheduler models the computation as an undirected graph $G = (V, E)$, where the set of vertices V consists of all slices. An edge $(u, v) \in E$ represents a mutual exclusion constraint signifying that slices u and v cannot be decoded concurrently. Each slice is characterized by a set of attributes that guide the scheduler's decisions:

a) *Neighbors:* Edges in the constraint graph are defined by the neighbors of each slice. A slice $S(t, p)$ can have up to six neighbors: two for temporal predecessor ($t - 1$) and successor ($t + 1$), and four spatial neighbors at time t . A data qubit slice always has at least one temporal dependency due to the syndrome stream. Spatial dependencies are introduced by multi-qubit lattice surgery.

b) *Decoding Status:* Each slice maintains a state transitioning through the automata depicted in Figure 8. A slice is initially **UNGENERATED**. Once its syndrome is produced by the quantum hardware, it becomes **PENDING**. A **PENDING**

slice is eligible for decoding, but may be blocked by a neighbor that is currently being decoded; in this case, it is marked as **OCCUPIED**. When a slice is ready and selected by the scheduler, it moves to **ASSIGNED** for the duration of its decoding. Upon successful processing, it enters **COMPLETED**. The Timeline dynamically maintains **PENDING** and **COMPLETED** slices during the online phase to facilitate the scheduler's decision.

c) *Causal Cone:* Slices associated with critical operations (i.e., Clifford corrections after a T-gate teleportation) have the causal cone as an additional attribute, which defines the complete set of historical slices that must be decoded to update the Pauli frame, as described in Section II-B2. Our framework employs a lazy computation strategy. When the scheduler requires a causal cone, it is calculated on-demand via a backward BFS starting from the critical slice's immediate spatial and temporal predecessors (i.e., the slices directly participating in the non-Clifford gate). The traversal only expands to (i) same-layer spatial neighbors and (ii) the one-step temporal predecessor at $t - 1$. A node in **COMPLETED** state will be pruned, while the BFS continues on the remaining frontier until the queue is exhausted. The results are then stored in a bounded LRU cache. All subsequent queries related to the same critical operation are served instantly from the cache.

Based on the slice abstraction, the compiler first lowers a high-level program to a Low-Level Instruction (LLI) stream over a 2-D logical-qubit layout. A single pass then builds the *Timeline* structure, where each unit stores: (i) an integer layer index t (unit: one syndrome-measurement cycle), (ii) spatial coordinate (r, c) , (iii) operation label, (iv) a 6-bit immediate-neighbor mask ($t-1, t+1, \uparrow, \downarrow, \leftarrow, \rightarrow$), (v) a deadline measured in *layers* to the nearest upcoming critical synchronization point (∞ if none), and (vi) a possible *causal cone* reference. During online simulation, this discrete timeline is mapped to scheduler time τ : syndrome-generation events occur at $\tau = 1, 2, \dots$ (one cycle step each), while decoding-completion events are scheduled at $\tau_{\text{finish}} = \tau_{\text{start}} + T_{\text{dec}}$.

2) *Online Phase: Real-time Execution:* During execution, the LLI is streamed to quantum hardware, which generates a continuous stream of syndrome data corresponding to the executed operations. This stream is the primary input to Triage, which dispatches decoding tasks to a shared pool of M physical decoders. The scheduler is triggered by two events: (i) arrival of a new syndrome layer, which introduces new pending slices; and (ii) completion of a decoding task, which releases decoder capacity.

At each syndrome-arrival event, the engine checks whether synchronization constraints are satisfied for critical operations. If satisfied, execution proceeds normally. If not, the engine inserts an *idle syndrome layer* at ℓ : all existing timeline layers with index $t \geq \ell$ are shifted to $t + 1$, and a new layer $t = \ell$ is created with per-qubit idle slices (while preserving spatial layout and carrying forward deadline semantics). This preserves causal order but delays all subsequent logical operations by one cycle. Decoders continue processing already assigned tasks asynchronously, and scheduling is retriggered after each decode completion and next syndrome arrival. Therefore, the

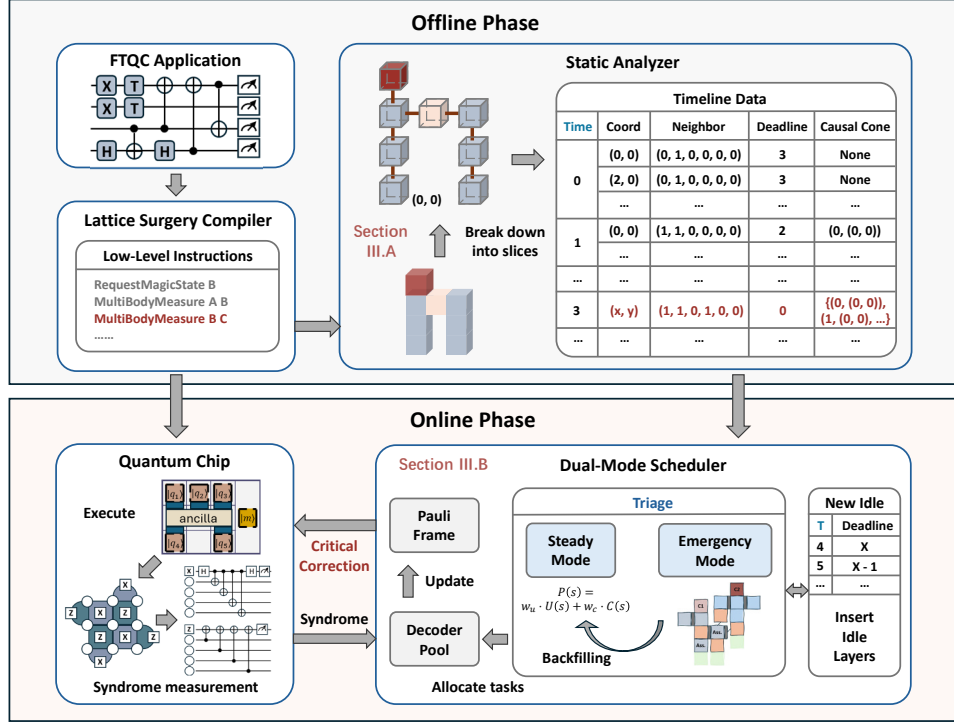


Fig. 7. Architectural Overview of the Triage Scheduling Framework. The offline phase consists of a compiler and a static analyzer that generate an annotated Timeline from LLIs. During the online phase, the scheduler uses this Timeline to dispatch a stream of syndrome data from the hardware to a finite pool of M physical decoders.

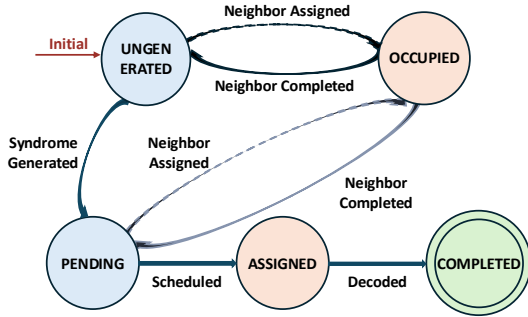


Fig. 8. State transition diagram for a slice's lifecycle.

scheduler objective is to maximize decoder throughput and minimize the number of such idle-layer insertions.

3) *Problem Formulation*: Now we can formally define our task as a real-time scheduling problem. Given:

- A shared pool of M decoders, each with speed r_{dec}^i relative to the syndrome generation speed.
- A dynamic, undirected constraint graph $G = (V, E)$ representing the set of all generated slices, slice attributes, and their mutually exclusive constraints.

The scheduler's task is to, at each decision point, find an assignment function $\pi : V' \rightarrow \{1, \dots, M\}$ where V' is a subset of all PENDING slices, satisfying:

- 1) The chosen set of slices V' must be an independent set in the graph G (i.e., for any two slices $u, v \in V'$, there is no edge between them).
- 2) The number of assigned slices cannot exceed the number of available decoders, $|V'| \leq M_{available}$.

The global objective is to produce a sequence of assignments that minimizes the total number of idle syndrome layers when synchronizing the Pauli frames of critical operations, thus minimizing the overall logical error rate.

B. The Dual-Mode Scheduling

Triage combines a lightweight *steady mode* for average-case throughput with a priority-aware *emergency mode* that resolves causal cones for imminent critical operations.

1) *Steady Mode: Heuristic Scheduling*: At each syndrome-generation or decode-completion event, the scheduler selects up to $M_{available}$ conflict-free PENDING slices using a priority function $P(V)$. We explore several heuristic policies: First-In-First-Out (FIFO) prioritizes slices with the oldest timestamp to clear backlogs chronologically; Earliest-Deadline-First (EDF) prioritizes slices with the smallest deadline to proactively service operations closest to becoming critical; and Min-Degree-First (MDF) prioritizes slices with the fewest neighbors to minimize decoding latency.

To balance these critical factors, we propose a unified priority function:

$$P(V) = w_u \cdot \text{Urgency}(V) + w_c \cdot \text{Cost-Efficiency}(V), \quad (2)$$

where w_u and w_c are tunable weighting factors ($w_u + w_c = 1$). The urgency term quantifies proximity to a critical deadline, defined as $\text{Urgency}(V) = 1/\text{Deadline}(V)$. The cost-efficiency term favors slices that are computationally cheaper to decode, defined by the inverse of the slice's degree, $\text{Cost-Efficiency}(V) = 1/(\text{Degree}(V) + 1)$.

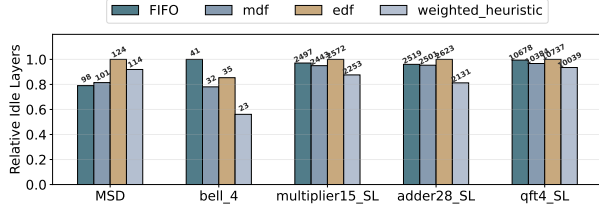


Fig. 9. Relative idle layers inserted by different heuristic policies. Heuristic-only scheduling leaves significant room for improvement.

Figure 9 provides a preliminary evaluation of these policies (detailed setups are deferred to Section IV). While our proposed weighted heuristic consistently outperforms the simple baselines, the sheer volume of idle layers remains substantial across all purely heuristic approaches. Pure heuristics inherently lack the foresight to guarantee low-latency Pauli frame updates for irregularly timed critical operations. This limitation motivates the emergency mode of our dual-mode architecture.

2) Emergency Mode: Predictive Causal Cone Coloring:

When the Triage Trigger signals an imminent deadline, the scheduler transitions to the emergency mode. Its objective is to resolve the causal cone of the impending critical operations with maximum parallelism, ensuring the necessary Pauli frames are updated before the Clifford correction executes.

Rather than making step-by-step decisions, the emergency mode employs a *predictive coloring* algorithm, detailed in Algorithm 1. This algorithm runs a discrete event simulation. It initializes a priority queue with only the PENDING slices in the on-demand causal cone, ensuring the input size minimal. The main loop advances a simulated clock to the next event and then greedily selects an independent set of tasks (Lines 10-14). The algorithm then records each selected slice in the final plan, and updates the auxiliary information (Lines 18-22). The core intuition is once inside the emergency mode, all slices in the causal cone share the same urgency. The primary factor for throughput is therefore the computational cost, resolved by the *MDF* policy. The online scheduler then transitions to a simple executor, dispatching the pre-computed tasks from the plan at their scheduled start times.

Complexity Analysis Let n be the number of slices in the causal cone. The initialization (Lines 4-7) pushes n elements into the priority queue Q , taking $O(n \log n)$ time. During the main loop, each slice is extracted and dispatched exactly once. Therefore, the inner loop (Lines 16-20) performs at most $6n$

Algorithm 1 Predictive Causal Cone Coloring

```

1: Input: Causal cone slice set  $C$ , current time  $t_{now}$ , decoder
   model  $D_{model}$ 
2: Output: An emergency plan  $P$ 
3: Initialize plan  $P \leftarrow \emptyset$ 
4: Initialize priority queue  $Q$ 
5: for slice  $s \in C$  do
6:    $s.t_{start} \leftarrow \max(t_{now}, s.t_{syndrome\_ready})$ 
7:   Push  $s$  to  $Q$ , prioritized by  $s.t_{start}$ 
8: end for
9: while  $Q$  is not empty do
10:   $t_{sim} \leftarrow \text{NextEvent}(Q, D_{model})$ 
11:   $R \leftarrow$  all slices from  $Q$  where  $s.t_{start} \leq t_{sim}$ 
12:  Sort  $R$  by degree
13:   $N_{free} \leftarrow D_{model}.num\_free(t_{sim})$ 
14:   $D_{dispatch} \leftarrow \text{SelectConflictFree}(R, N_{free})$ 
15:  for slice  $s \in D_{dispatch}$  do
16:    Add  $(t_{sim}, s)$  to  $P$ 
17:     $t_{fin} \leftarrow t_{sim} + \text{CalculateDuration}(s.degree)$ 
18:    for neighbor  $n \in Q$  of  $s$  do
19:       $n.t_{start} \leftarrow \max(n.t_{start}, t_{finish})$ 
20:       $n.degree \leftarrow n.degree - 1$ 
21:      Update position of  $n$  in  $Q$ 
22:    end for
23:  end for
24:  Re-insert non-dispatched slices from  $R$  back into  $Q$ 
25: end while
26: return  $P$ 

```

neighbor updates, with each priority queue position update taking $O(\log n)$. Sorting the ready set R (Line 10) is also bounded by $O(n \log n)$. Thus, the overall worst-case complexity scales efficiently at $O(n \log n)$. We will empirically validate this overhead in Section V-E.

3) *The Triage Trigger*: The Triage Scheduler's adaptivity and efficiency is governed by the *Triage trigger*, the mechanism that decides precisely when and how to transition to the emergency mode. The trigger is activated whenever any PENDING slice's deadline reaches a predefined threshold $\tau_{emergency}$ (e.g., $\tau_{emergency} = 4$). To prevent the scheduling complexity from causing latency spikes on exceptionally large causal cones (which often accumulate near the end of highly entangled applications), we enforce a strict $\text{ScopeCap} < 100$. If an evaluated causal cone exceeds this size limit, the scheduler falls back to the steady mode.

To avoid thrashing, Triage re-plans only when all expansion-driven conditions hold:

- The set of urgent slices introduces a causal cone that is not fully contained within the currently emergency scope.
- The expansion is significant, exceeding a defined fraction (e.g., 30%) of the existing scope's size.
- A minimum time interval (e.g., 2) has passed since the last re-plan.

When expansion-driven conditions are met and there is overlap between the two scopes, the scheduler performs an

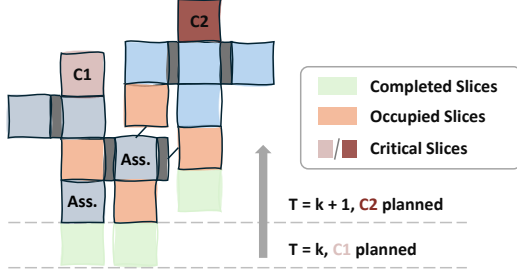


Fig. 10. A 2-D snapshot of the Triage Trigger's operation. At $T = k + 1$, a new critical slice C_2 triggers a scope expansion.

incremental update. Figure 10 provides a 2-D simplified snapshot of this process. The scheduler operates on an evolving dependency graph. At time $T = k$, an emergency plan for a critical slice C_1 is already active. As time goes to $T = k + 1$, a new critical slice C_2 becomes urgent, triggering a re-evaluation. The new emergency scope for C_2 excludes already COMPLETED slices, and the incremental planner will take into account the blocking effect of C_1 's plan on future slices.

Critical-path impact of scheduling. The part of Triage that can introduce noticeable latency overhead is emergency-mode causal-cone planning. Triage does not assume that every scheduling computation stalls the quantum processor. An emergency plan is cached and subsequently executed as a lightweight dispatch table. Therefore, only the portion of planning, dispatch, or interconnect latency that cannot be hidden behind ongoing decoding can affect the critical path. In Section V-E, we conservatively model this unhidden latency by delaying task start times.

4) *Throughput Maximization via Opportunistic Backfilling:* While the emergency mode is latency-optimal, its resource utilization can be inefficient. The parallelism of a causal cone often dictates a peak decoder requirement, M_{peak} , that is less than the total available decoders, M . As illustrated in Figure 11, this discrepancy creates idle decoders and wastes computational resources. To reclaim this lost throughput, we introduce an *opportunistic backfilling* mechanism. The scheduler first computes M_{peak} from the emergency plan, then derives the *max usable decoders* for backfilling at each pass as $M_{usable}(t) = \max(0, \min(M - M_{peak} - B_{bf}(t), F(t) - E(t)))$, where $B_{bf}(t)$ is currently running backfill tasks, $F(t)$ is physically free decoders, and $E(t)$ is emergency tasks dispatched in the same pass. This budget is then used to dispatch non-critical, causally-disconnected tasks using the heuristic scheduler, thereby maximizing throughput without any risk of interfering with the critical emergency plan.

IV. EXPERIMENT SETUP

Simulation Framework. We develop a simulation framework that models the entire classical control pipeline: The compiler emits LLIs, the static analyzer constructs an annotated Timeline, and a discrete-event simulator generates

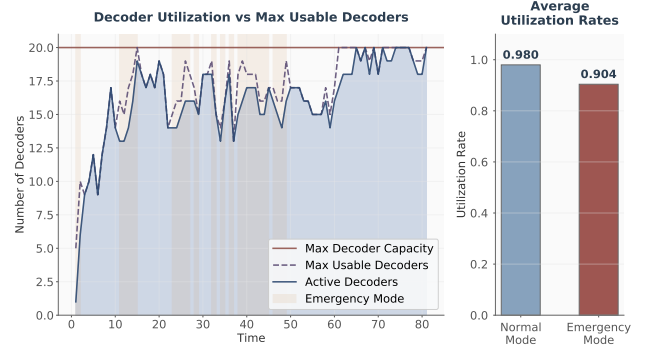


Fig. 11. Motivation for Opportunistic Backfilling. Left: Triage's decoder utilization over time showing active decoders (blue), maximum capacity (purple dashed), and emergency periods (orange). Right: Comparison of utilization rates. The utilization can be improved by backfilling.

syndromes and invokes the scheduler on syndrome arrivals and task completions. Before each critical operation, it checks whether the causal cone is decoded; otherwise it inserts an idle syndrome layer into the Timeline, and generates a layer of syndrome similar to the memory experiment [10]. To prevent an unrecoverable backlog of decoding tasks, the simulation is forcibly terminated if the total number of inserted idle layers exceeds ten times the original layer count of the benchmark. The scheduler is invoked on every syndrome generation and every task completion.

Metrics. We evaluate scheduler performance using two metrics. Since an idle layer is inserted only when synchronization fails, we measure *the number of inserted idle layers* as a direct metric for the scheduler's ability to handle critical operations. The simulation will also terminate when a significant backlog is detected. The *logical error rate (LER)* provides the ultimate measure which is correlated to the total execution layers. We first simulate window-based lattice surgery using a circuit-level noise model, and then aggregate the LER of each layer to obtain the overall LER.

System Configuration. We use a Litinski-style compiler [38] to generate LLIs for our benchmarks. The instruction set is composed of multi-patch measurement, patch rotation and idle. To model the decoding time, we profiled the `pymatching` decoder [16] on varying decoding volume, fitting the empirical data to a power-law model: $t_{decode} = A \cdot (\text{volume})^\alpha$. Given $\alpha = 1.17$, our framework's decoding time for a given slice is determined by the size of its window buffer, which is directly related to the number of unresolved neighbors (i.e., its degree in the constraint graph). Note that our assumption that latency is monotonically increasing with volume holds for any practical decoder, so the relative performance trends in our evaluation are expected to be general. Pattern-dependent runtime variation is modeled separately in Section V-D, where a calibrated heavy-tail jitter model is injected into every decoder task.

For Monte Carlo, we simulate a $d = 9$ rotated surface code under circuit-level depolarizing noise at $p = 3 \times 10^{-3}$, and

TABLE I
CHARACTERISTICS OF THE FTQC BENCHMARK SUITE.

Benchmark	Short Name	# LQubits	# Layers	# T-Gates	T-Den.*	Category
T-State Injection	T_injection	9	13	1	7.69%	FT Gadget
Arbitrary Rotation ($\pi/7$)	rotation_C+T	1	2694	318	11.80%	FT Benchmark
Magic State Distillation 15-to-1	MSD15to1	5	24	11	45.83%	FT Gadget
Bell State Preparation	bell4	4	41	5	12.20%	FT Gadget
15-qubit Multiplier	mult15_CL*	15	586	252	43.00%	Arithmetics
15-qubit Multiplier	mult15_SL*	15	508	252	49.61%	Arithmetics
28-qubit Adder	adder28_CL	28	1894	168	8.87%	Arithmetics
28-qubit Adder	adder28_SL	28	640	168	26.25%	Arithmetics
64-bit Adder	adder64_SL	64	1492	392	26.27%	Arithmetics
118-bit Adder	adder118_SL	118	2770	728	26.28%	Arithmetics
11-qubit SECA	seca11_SL	11	140	56	40.00%	Arithmetics
4-qubit Variational	variational4_SL	4	3636	402	11.06%	Variational Algorithm
4-qubit QFT	qft4_SL	4	1505	459	30.50%	QFT Algorithm
4-qubit Trotterization	trotter4_SL	4	2198	576	26.21%	Hamiltonian Simulation
26-qubit Ising Model	ising26_SL	26	11303	3688	32.63%	Hamiltonian Simulation

* T-Den.: The proportion of T gates. CL: Compiled with Compact Layout [38]. SL: Compiled with Standard Layout [44].

we extrapolate the logical error rate to the $d = 21$ case. We perform the Monte Carlo simulations with Stim [42], with each point made of at least 10^5 runs.

Benchmarks. To cover a wide range of scenarios, we select a series of benchmarks from QASMBench [43] with various T-gate densities. Furthermore, to demonstrate the universality of our framework, we include compiled versions for both Compact Layout (CL) [38] and Standard Layout (SL) [44]. A summary of these benchmarks is provided in Table I.

Simulation Device. All experiments were conducted with an Intel i9-14900K processor and 188 GB of RAM. The simulation framework was implemented in Python 3.9.

Baselines. While parallel window decoding has been extensively discussed [24]–[26], a framework for fine-grained scheduling has yet to be established. We construct baselines within our framework in Section III-A to demonstrate the benefits of our spatio-temporal parallelism:

- *Serial sliding window* [41]: A scheduler processes a block of slices involved in a lattice surgery operation at a time, but does not process slices at later times in advance.
- *Time-parallel window* [24]: A scheduler leverages parallelism across the time dimension for logical patches, but does not split up multi-qubit operations.
- *SWIPER* [26]: A state-of-the-art speculative scheduler. We reproduce its *successor-based* strategy which is optimistic regarding mis-speculations, setting 10% mis-prediction rate and 10% speculation time. Furthermore, the speculative decoding module is not included in the decoder usage.

V. EVALUATION RESULTS

We design our evaluation based on the following key questions to reflect the practicality of our framework.

Q1 How does the spatio-temporal parallelism compare to default and SOTA strategies under varying constraints?

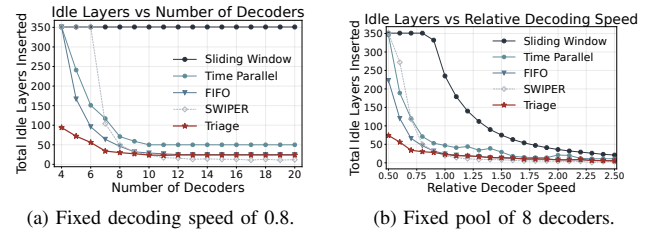


Fig. 12. Relation between idle layers inserted and (a) number of available decoders; (b) relative decoding speed (τ_{dec}/τ_{gen}).

- Q2** How do the proposed schedulers perform across a diverse suite of FTQC applications in terms of idle reduction and logical error rates?
- Q3** How resilient is the Triage scheduler to real-world decoder latency fluctuations?
- Q4** What are the computational overheads of the proposed schedulers? Does Triage’s advantage degenerate when considering scheduling and interconnect latency?
- Q5** How do internal mechanisms and hyperparameters contribute to the overall performance?

In the following simulations, the heuristic weights are set to $w_u = w_c = 0.5$, the Triage trigger’s replan scope threshold is 0.3, and the minimum planning interval is 2.

A. Motivating Spatio-Temporal Parallelism

We first evaluate five schedulers representing a spectrum of parallelization strategies on the Bell4 on Litinski’s compact layout. This task comprises 39 logical layers and includes 5 critical $\pi/8$ gates. The schedulers under comparison are: the baseline *sliding window* and *time-parallel* schedulers; the speculative scheduler *SWIPER* [26]; our *time-space-parallel* scheduler with FIFO policy; and our *Triage* scheduler. Figure 12 shows the number of inserted idle layers as we vary the number of available decoders and the relative speed of each individual decoder.

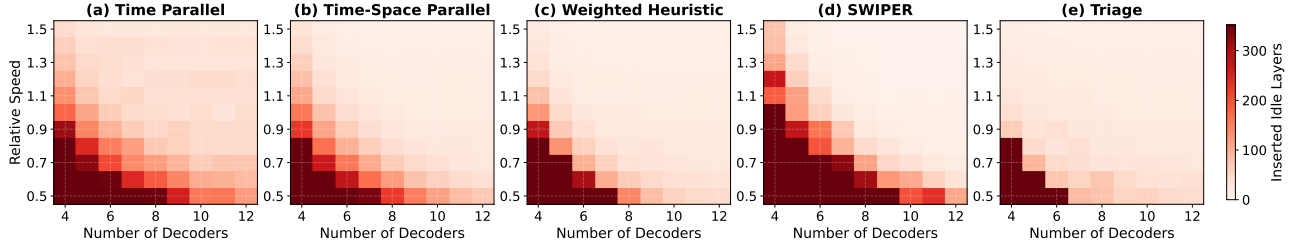


Fig. 13. Heatmaps illustrating the number of inserted idle layers for different schedulers across various decoder counts and relative speeds. Darker red indicates a higher number of idle layers, signifying worse performance. The *Triage* scheduler consistently achieves near-best performance across the entire space and defines the performance frontier in resource-constrained scenarios.

a) *Observation 1: Serial processing is fundamentally unscalable:* As shown in both figures, the sliding window scheduler exhibits the worst performance. The flat line demonstrates that its sequential nature makes it fundamentally unable to leverage parallel hardware resources to increase throughput.

b) *Observation 2: Spatio-temporal parallelism enables superior resource utilization:* The *time-parallel* scheduler offers a significant improvement over the serial approach, but its performance saturates at a high number of idle layers. The *time-parallel* scheduler is bottlenecked by its inability to break down correlated multi-qubit operations, resulting in a high floor. In contrast, the *time-space-parallel* schedulers can process these complex operations with a much finer granularity, achieving a lower saturation point.

c) *Observation 3: Triage outperforms SWIPER under resource constraints:* *SWIPER* leverages its speculative mechanism to achieve extremely high parallelism, showing competitive performance when resources are abundant. However, these advantages diminish significantly in resource-constrained regimes where speculative overheads can lead to resource contention. In contrast, our *Triage* scheduler demonstrates superior performance in these scenarios.

B. Design Space Exploration of Scheduling Strategies

To characterize the performance landscape of various scheduling strategies, we conduct a design space exploration across a wide range of decoder counts and relative speeds, comparing our *Triage* scheduler against the *Time Parallel* scheduler, the *Time-Space Parallel* (FIFO) scheduler, and the *SWIPER* [26]. Figure 13 presents the performance of these schedulers as heatmaps, where darker regions indicate a higher number of inserted idle layers and thus poorer performance. The *Triage* scheduler’s performance is particularly pronounced in the most challenging regions where decoders are both slow (low y-axis value) and scarce (low x-axis value), whereas *SWIPER* achieves the global minimum of idle layers when decoding resources are relatively abundant.

Figure 14 synthesizes these results into an optimal map. Each cell is colored to indicate which scheduler achieved the best performance for that specific resource configuration. The *Triage* scheduler (red) defines most of the feasible resource-constrained lower-left frontier. In contrast, *SWIPER* (light blue) tends to be optimal in the resource-abundant upper-right

regime. Notably, the lower-left black regions denote a regime of failure where extreme resource scarcity forces all schedulers to trigger the backlog-induced termination threshold.

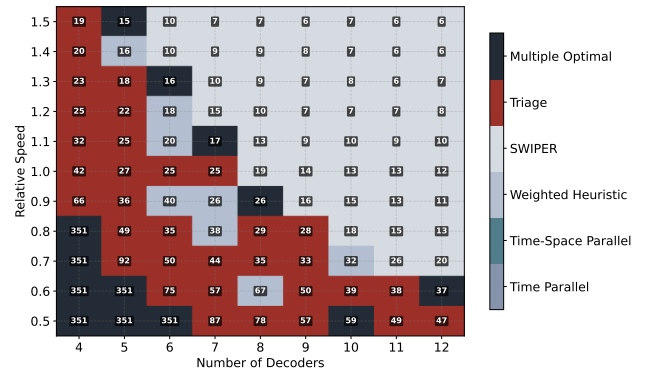


Fig. 14. The optimal scheduler map on the Bell4 application. Each cell in the grid is colored according to the best-performing scheduler for that decoder pool configuration.

C. Performance Across Benchmarks

We now evaluate the schedulers on various FTQC benchmarks. Figure 15 illustrates the idle layers inserted and LER across all benchmarks in two representative resource scenarios: a **Parallelism-Rich Scenario**, featuring numerous but slow decoders (count = $2 \times \#LQs$, speed=0.9), and a **Latency-Rich Scenario**, featuring few but fast decoders (count = $\#LQs$, speed=1.8). The height of the idle-step bars is normalized within each application, while the absolute values are labeled.

The results in both Figure 15a and Figure 15b show a trend of hierarchy of performance. For visual clarity, *SWIPER* is omitted as its performance under resource-constrained scenarios is comparable to the FIFO policy. The *time-only parallelism* scheduler performs the worst, while the *Triage* scheduler consistently achieves the best or near-best performance. The FIFO policy itself is not particularly bad, as starting from the bottom of the timeline results in most allocated slices having small degrees. Across these benchmarks, *Triage* achieves an average logical error rate reduction of 52.6% compared to the time-parallel baseline.

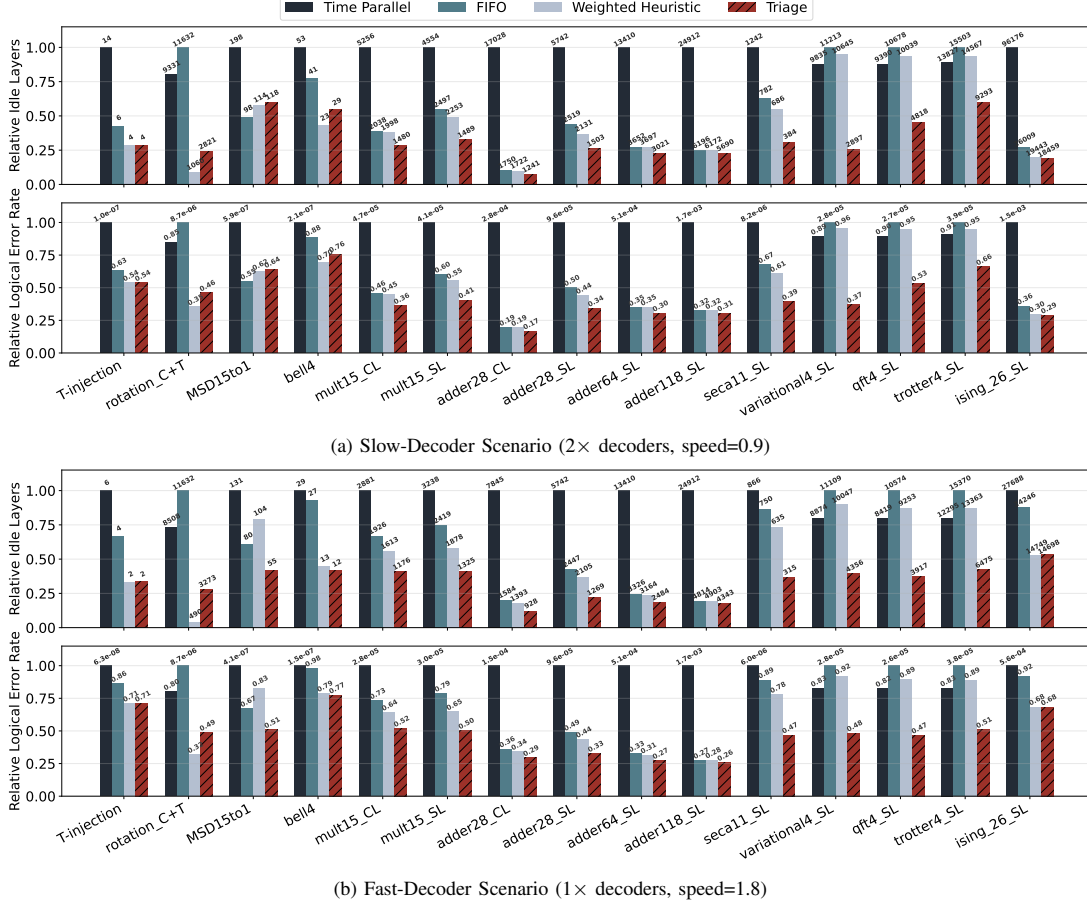


Fig. 15. LER comparison across all benchmarks for (a) a resource-constrained scenario and (b) a resource-abundant scenario. Lower bars indicate better performance. Triage outperforms the baseline in nearly all cases.

An intriguing exception is the variational algorithm, where time-space parallel scheduling using a single strategy mode performs worse than time-only parallel scheduling alone. In this case, splitting the multi-qubit logical operations into dependent slices increases scheduling difficulty, yet the performance of the Triage scheduler remains superior.

Estimation of Physical Execution Time. The total wall-clock time is determined by the total number of layers (including inserted idles) and the duration of each layer: $T_{total} = N_{total_layers} \times T_{layer}$. Each logical layer in a surface code typically requires d rounds of syndrome measurements, so $T_{layer} = d \times T_{meas}$. For a distance $d = 21$ code, T_{layer} varies significantly across platforms: approximately $21 \mu s$ for superconducting qubits ($T_{meas} \approx 1 \mu s$), and ranging from 2.1 ms to 21 ms for ion traps or neutral atoms. By reducing the number of idle layers our Triage scheduler translates directly into significant wall-clock time savings.

D. Impact of Stochastic Decoding Latency

In practical FTQC systems, decoding latency is not deterministic but fluctuates due to varying error patterns. We model

decoder latency jitter as a mean-preserving lognormal factor,

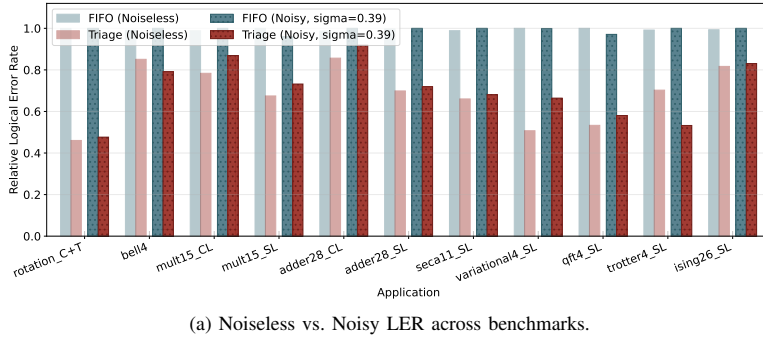
$$t_{actual} = t_{estimated} \cdot \exp\left(-\frac{\sigma^2}{2} + \sigma z\right), \quad z \sim \mathcal{N}(0, 1), \quad (3)$$

so that the mean latency remains consistent while introducing a heavy tail characteristic of real-time systems. The jitter scale σ is parameterized as

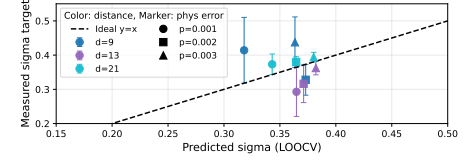
$$\sigma(d, p) = \text{clamp}(\sigma_{base} + \alpha_d \log_2(d/5) + \alpha_p(p - p_{ref}), \sigma_{min}, \sigma_{max}) \quad (4)$$

Here, σ_{base} is the baseline jitter at $(d = 5, p = p_{ref})$, α_d captures distance-driven complexity growth, and α_p captures error-rate-driven complexity growth.

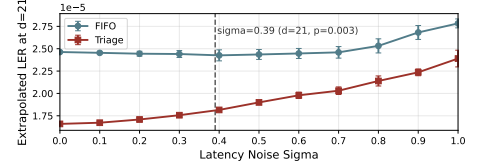
The calibration set is built from per-shot `pymatching` latency measurements on Stim-generated rotated surface-code circuits with 15K measured shots per setting after warmup. We obtain $\sigma_{base} = 0.3447$, $\alpha_d = 0.0041$, $\alpha_p = 15.03$, $p_{ref} = 10^{-3}$, $\sigma_{min} = 0.30$, $\sigma_{max} = 0.70$. Leave-one-out validation predicts the held-out σ with a mean absolute error of 0.064 and captures tail quantiles with about 15% relative error. Thus, the lognormal model is a calibrated heavy-tail



(a) Noiseless vs. Noisy LER across benchmarks.



(b) Empirical Lognormal σ : Measured vs. Predicted.



(c) LER sensitivity to σ (Multiplier15_SL).

Fig. 16. Evaluation of the Triage scheduler under stochastic latency. (a) compares LER in noiseless and noisy scenarios; (b) validates our $\sigma(d, p)$ fit against empirical measurements; (c) demonstrates Triage’s robustness as latency jitter increases.

service-time abstraction used to test whether Triage remains robust when complex syndrome patterns create tail latency.

As shown in Figure 16b, our lognormal model closely matches the empirical σ measured from `pymatching`. Figure 16a presents a detailed comparison of LER for each application under both noiseless and noisy environments. Although the presence of stochastic latency inevitably leads to a higher LER, *Triage* consistently maintains a significant advantage over the baseline. This robustness is further quantified in Figure 16c, which tracks the LER of the `Multiplier15_SL` benchmark as the jitter intensity σ varies from 0 to 1. While the gap between FIFO and Triage narrows at extreme noise levels, *Triage* remains the superior strategy.

E. Computational Overhead Analysis

Figure 17a breaks down the runtime into total scheduling time and average latency per logical layer. Note that all reported runtimes strictly isolate the pure algorithmic latency required to generate a scheduling plan, excluding the simulation environment. The measured wall-clock numbers characterize our Python prototype. Triage has sub-millisecond median per-layer scheduling cost across our benchmarks, but large emergency scopes can create multi-millisecond tail latency. This is acceptable for slow-cycle platforms such as ion traps or neutral atoms, but a superconducting implementation with $\sim 20 \mu\text{s}$ logical-layer cycles would require a compiled or hardware-assisted implementation. Our claim is that the algorithmic structure is bounded: emergency planning scales as $O(n \log n)$ and *ScopeCap* prevents pathological causal cones from entering the critical path.

To rigorously assess how this scheduler latency impacts overall system performance, we conduct a sensitivity simulation that incorporates real-time scheduling delays. We define a *Delay Ratio* as the baseline FIFO scheduler runtime relative to the decoding time, sweeping this ratio from 0.00 to 0.20. For heuristic policies, task assignment is delayed proportionally. For *Triage*’s emergency mode, the delay is dynamically calculated using the fitted $O(n \log n)$ function

based on the real-time scope size. As shown in Figure 17c with the `Multiplier15_SL` benchmark, the *Triage without ScopeCap* policy suffers from severe runtime penalties when attempting to resolve massive causal cones near the end of applications, causing the system to hit a backlog failure at a delay ratio of 0.06. By enforcing the *ScopeCap*, *Triage* maintains robust and superior performance across the entire delay spectrum.

F. Sensitivity and Ablation Studies

1) *Impact of Decoding Window Size*: The decoding window size presents a critical trade-off between decoding throughput and individual operation fidelity. Smaller windows accelerate processing but reduce syndrome context, whereas larger windows improve accuracy at the cost of increased latency. This balance is dictated by classical resource availability.

Figure 18 illustrates this trade-off under two regimes: resource-constrained (speed = $0.8\times$) and resource-rich (speed = $1.5\times$). In the constrained regime, smaller buffers are optimal as they maintain high decoding throughput, thereby minimizing total idle layers and the resulting aggregate LER. Conversely, in resource-rich scenarios, the bottleneck shifts from throughput to individual operation fidelity, favoring larger windows. Consequently, the optimal buffer size is hardware-dependent; while current latency-limited systems necessitate smaller buffers, future high-performance hardware will likely favor larger windows approaching $d/2$ [27].

2) *Impact of Hyperparameters*: We evaluate *Triage*’s sensitivity to its primary hyperparameters: the heuristic weight (w_u) and the emergency threshold ($\tau_{\text{emergency}}$). As Figure 19(a) shows, sweeping w_u from 0 to 1 reveals that the logical error rate is remarkably robust, eliminating the need for application-specific tuning. Similarly, Figure 19(b) demonstrates stable performance across a moderate threshold range ($\tau_{\text{emergency}} \in [2, 8]$). Performance degrades only when excessively high thresholds (e.g., $\tau_{\text{emergency}} = 16$) delay necessary interventions during congestion spikes. Overall, *Triage* exhibits strong robustness to parameter variations without requiring fine-grained tuning.

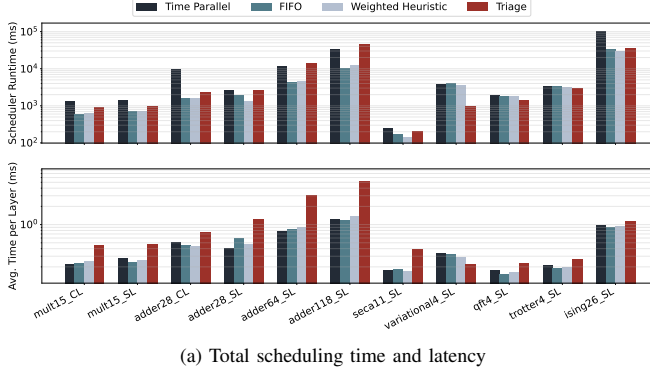


Fig. 17. Computational overhead analysis. (a) Total scheduling time per application (top) and average latency per logical layer (bottom). (b) Plan schedule time versus emergency scope size, best captured by an $O(n \log n)$ fit ($y = a \cdot n \log n$, $a = 0.01513$, $R^2 = 0.805637$). (c) System performance (Idle Layers) under simulated scheduler latency.

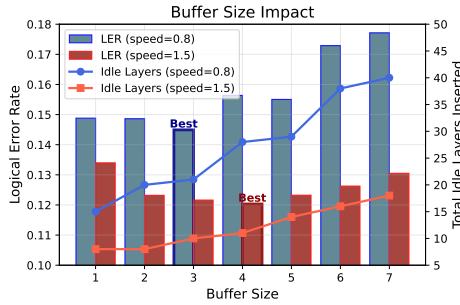


Fig. 18. LER (bars) and inserted idle layers (lines) as a function of the window decoding buffer size with Triage at $d = 7$. The decoder count is fixed to 8. An optimal operating point appears around a buffer size of $d/2$.

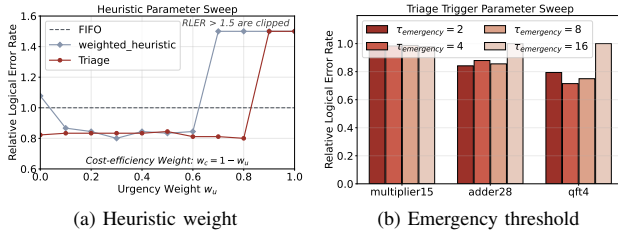


Fig. 19. Sensitivity analysis of *Triage*. Triage has robust performance across a wide range of parameter configurations.

VI. RELATED WORK

Improvements on Decoders. Research on decoders for FTQC has focused on improving accuracy, latency, and scalability. This includes algorithmic approaches, such as lookup table decoders [20], [45], [46], minimum-weight perfect matching (MWPM) decoders for surface codes [41], [47], [48]. In addition, system-level approaches have been investigated to reduce decoding latency [22], [49]–[52]. These include specialized solutions for superconducting qubits [16], [21], [53], [54], hierarchical decoders [55], optimized union-find decoders [56], and FPGA-based implementations [57], [58].

These individual-decoder efforts complement our framework, which schedules a shared decoder pool to manage system-wide latency constraints.

Decoder Scheduling. Most existing works on decoder design assume a dedicated decoder is statically allocated for each logical qubit [24], [25], [59]. Recent concurrent works have begun to address the challenges of dynamic decoder scheduling [26], [28], [59], with *SWIPER* [26] representing the current SOTA. Our framework focuses on mitigating the decoding pressure induced by non-Clifford operations, achieving lower logical error rates under resource-constrained scenarios. Furthermore, our simulation enables a direct evaluation of how classical resource bottlenecks dictate final performance.

Compilers for Optimizing Lattice Surgery. Many compilers have been proposed to improve the scheduling of lattice surgery operations [60]–[65], and several works have also focused on increasing the parallelism during these procedures [44], [66]–[68]. In our study, we use the compiler introduced in [62], [63] to compare different strategies for decoder scheduling. Integrating advanced compiler techniques may further improve overall performance.

VII. CONCLUSION

In this work, we identified the management of classical decoder resources as a bottleneck for scalable FTQC. We utilized a spatio-temporal framework to formulate the constrained dynamic scheduling problem. We then proposed *Triage*, a dual-mode scheduling architecture that maximizes resource utilization. Our implementation focused on surface codes, the principles of constrained parallel-window scheduling are broadly relevant. Extending this framework to general QLDPC codes will be a promising future work. Furthermore, exploring the co-design of the quantum compiler and scheduler represents a next step, enabling the compiler to optimize circuits with classical resource awareness.

ACKNOWLEDGMENT

We would like to thank the anonymous reviewers for their helpful feedback and suggestions.

REFERENCES

- [1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [2] A. M. Childs, D. Maslov, Y. Nam, N. J. Ross, and Y. Su, "Toward the first quantum simulation with quantum speedup," *Proceedings of the National Academy of Sciences*, vol. 115, no. 38, pp. 9456–9461, 2018.
- [3] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.
- [4] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
- [5] A. A. Clerk, M. H. Devoret, S. M. Girvin, F. Marquardt, and R. J. Schoelkopf, "Introduction to quantum noise, measurement, and amplification," *Reviews of Modern Physics*, vol. 82, no. 2, pp. 1155–1208, 2010.
- [6] E. T. Campbell, B. M. Terhal, and C. Vuillot, "Roads towards fault-tolerant universal quantum computation," *Nature*, vol. 549, no. 7671, pp. 172–179, 2017.
- [7] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 3, p. 032324, 2012.
- [8] A. J. Landahl, J. T. Anderson, and P. R. Rice, "Fault-tolerant quantum computing with color codes," *arXiv preprint arXiv:1108.5738*, 2011.
- [9] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, no. 8005, pp. 778–782, 2024.
- [10] Google Quantum AI and Collaborators, "Quantum error correction below the surface code threshold," *Nature*, vol. 638, no. 8052, pp. 920–926, 2025.
- [11] D. Bluvstein, S. J. Evered, A. A. Geim, S. H. Li, H. Zhou, T. Manovitz, S. Ebadi, M. Cain, M. Kalinowski, D. Hangleiter *et al.*, "Logical quantum processor based on reconfigurable atom arrays," *Nature*, vol. 626, no. 7997, pp. 58–65, 2024.
- [12] K. Wang, Z. Lu, C. Zhang, G. Liu, J. Chen, Y. Wang, Y. Wu, S. Xu, X. Zhu, F. Jin *et al.*, "Demonstration of low-overhead quantum error correction codes," *arXiv preprint arXiv:2505.09684*, 2025.
- [13] L. Caune, L. Skoric, N. S. Blunt, A. Ruban, J. McDaniel, J. A. Valery, A. D. Patterson, A. V. Gramolin, J. Majaniemi, K. M. Barnes *et al.*, "Demonstrating real-time and low-latency quantum error correction with superconducting qubits," *arXiv preprint arXiv:2410.05202*, 2024.
- [14] A. Eickbusch, M. McEwen, V. Sivak, A. Bourassa, J. Atalaya, J. Claes, D. Kafri, C. Gidney, C. W. Warren, J. Gross *et al.*, "Demonstrating dynamic surface codes," *arXiv preprint arXiv:2412.14360*, 2024.
- [15] B. M. Terhal, "Quantum error correction for quantum memories," *Reviews of Modern Physics*, vol. 87, no. 2, pp. 307–346, 2015.
- [16] O. Higgott and C. Gidney, "Sparse Blossom: correcting a million errors per core second with minimum-weight matching," *Quantum*, vol. 9, p. 1600, 2025.
- [17] N. Delfosse, V. Londe, and M. E. Beverland, "Toward a Union-Find decoder for quantum ldpc codes," *IEEE Transactions on Information Theory*, vol. 68, no. 5, pp. 3187–3199, 2022.
- [18] J. Chen, Z. Yi, Z. Liang, and X. Wang, "Improved belief propagation decoding algorithms for surface codes," *IEEE Transactions on Quantum Engineering*, 2025.
- [19] T. Müller, T. Alexander, M. E. Beverland, M. Bühler, B. R. Johnson, T. Maurer, and D. Vandeth, "Improved belief propagation is sufficient for real-time decoding of quantum memory," *arXiv preprint arXiv:2506.01779*, 2025.
- [20] P. Das, A. Locharla, and C. Jones, "LILLIPUT: a lightweight low-latency lookup-table decoder for near-term quantum error correction," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 541–553.
- [21] S. Vittal, P. Das, and M. Qureshi, "Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.
- [22] N. Alavisamani, S. Vittal, R. Ayanzadeh, P. Das, and M. Qureshi, "Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, 2024, pp. 818–833.
- [23] Y. Wu, N. Liyanage, and L. Zhong, "Micro Blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, 2025, pp. 639–654.
- [24] L. Skoric, D. E. Browne, K. M. Barnes, N. I. Gillespie, and E. T. Campbell, "Parallel window decoding enables scalable fault tolerant quantum computation," *Nature Communications*, vol. 14, no. 1, p. 7040, 2023.
- [25] X. Tan, F. Zhang, R. Chao, Y. Shi, and J. Chen, "Scalable surface-code decoders with parallelization in time," *PRX Quantum*, vol. 4, no. 4, p. 040344, 2023.
- [26] J. Viszlai, J. D. Chadwick, S. Joshi, G. S. Ravi, Y. Li, and F. T. Chong, "SWIPER: Minimizing fault-tolerant quantum program latency via speculative window decoding," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1386–1401.
- [27] S. F. Lin, E. C. Peterson, K. Sankar, and P. Sivarajah, "Spatially parallel decoding for multi-qubit lattice surgery," *Quantum Science and Technology*, vol. 10, no. 3, p. 035007, 2025.
- [28] S. Maurya and S. Tannu, "Managing classical processing requirements for quantum error correction," *arXiv preprint arXiv:2406.17995*, 2024.
- [29] L. Riesebo, X. Fu, S. Varsamopoulos, C. G. Almudever, and K. Bertels, "Pauli Frames for quantum computer architectures," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017, pp. 1–6.
- [30] D. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, "Surface code quantum computing by lattice surgery," *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012.
- [31] A. G. Fowler and C. Gidney, "Low overhead quantum computation using lattice surgery," *arXiv preprint arXiv:1808.06709*, 2018.
- [32] N. de Beaudrap and D. Horsman, "The ZX calculus is a language for surface code lattice surgery," *Quantum*, vol. 4, p. 218, 2020.
- [33] S. Bravyi and A. Kitaev, "Universal quantum computation with ideal Clifford gates and noisy ancillas," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 71, no. 2, p. 022316, 2005.
- [34] D. Litinski, "Magic state distillation: Not as costly as you think," *Quantum*, vol. 3, p. 205, 2019.
- [35] S. Bravyi and J. Haah, "Magic-state distillation with low overhead," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 5, p. 052329, 2012.
- [36] C. Jones, "Multilevel distillation of magic states for quantum computing," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 87, no. 4, p. 042305, 2013.
- [37] C. Gidney, N. Shutty, and C. Jones, "Magic state cultivation: growing t states as cheap as cnot gates," *arXiv preprint arXiv:2409.17595*, 2024.
- [38] D. Litinski, "A game of surface codes: Large-scale quantum computing with lattice surgery," *Quantum*, vol. 3, p. 128, 2019.
- [39] D. P. DiVincenzo and P. Aliferis, "Effective fault-tolerant quantum computation with slow measurements," *Physical review letters*, vol. 98, no. 2, p. 020501, 2007.
- [40] T. Itogawa, Y. Takada, Y. Hirano, and K. Fujii, "Efficient magic state distillation by zero-level distillation," *PRX Quantum*, vol. 6, no. 2, p. 020356, 2025.
- [41] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.
- [42] C. Gidney, "Stim: a fast stabilizer circuit simulator," *Quantum*, vol. 5, p. 497, 2021.
- [43] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.
- [44] Y. Hirano and K. Fujii, "Locality-aware Pauli-based computation for local magic state preparation," *arXiv preprint arXiv:2504.12091*, 2025.
- [45] Y. Tomita and K. M. Svore, "Low-distance surface codes under realistic quantum noise," *Physical Review A*, vol. 90, no. 6, p. 062320, 2014.
- [46] C. Ryan-Anderson, J. G. Bohnet, K. Lee, D. Gresh, A. Hankin, J. Gaebler, D. Francois, A. Chernoguzov, D. Lucchetti, N. C. Brown *et al.*, "Realization of real-time fault-tolerant quantum error correction," *Physical Review X*, vol. 11, no. 4, p. 041058, 2021.

- [47] A. G. Fowler, "Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $o(1)$ parallel time," *arXiv preprint arXiv:1307.1740*, 2013.
- [48] O. Higgott, "PyMatching: A python package for decoding quantum codes with minimum-weight perfect matching," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–16, 2022.
- [49] J. Kim, D. Min, J. Cho, H. Jeong, I. Byun, J. Choi, J. Hong, and J. Kim, "A fault-tolerant million qubit-scale distributed quantum computer," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 1–19.
- [50] S. Vittal, P. Das, and M. Qureshi, "ERASER: Towards adaptive leakage suppression for fault-tolerant quantum computing," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 509–525.
- [51] W. Liao, Y. Suzuki, T. Tanimoto, Y. Ueno, and Y. Tokunaga, "WIT-Greedy: hardware system design of weighted iterative greedy decoder for surface code," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*, 2023, pp. 209–215.
- [52] P. Thantharate and A. Thantharate, "Q-CODA: Co-designing quantum codes and architectures for hardware-aware quantum error correction," in *International Symposium on Quantum Sciences: Applications and Challenges*, 2023, pp. 134–151.
- [53] A. Holmes, M. R. Jokar, G. Pasandi, Y. Ding, M. Pedram, and F. T. Chong, "NISQ+: Boosting quantum computing power by approximating quantum error correction," in *2020 ACM/IEEE 47th annual international symposium on computer architecture (ISCA)*. IEEE, 2020, pp. 556–569.
- [54] Y. Ueno, M. Kondo, M. Tanaka, Y. Suzuki, and Y. Tabuchi, "QECool: On-line quantum error correction with a superconducting decoder for surface code," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 451–456.
- [55] N. Delfosse, "Hierarchical decoding to reduce hardware requirements for quantum computing," *arXiv preprint arXiv:2001.11427*, 2020.
- [56] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, "AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 259–273.
- [57] N. Liyanage, Y. Wu, A. Deters, and L. Zhong, "Scalable quantum error correction for surface codes using FPGA," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 916–927.
- [58] N. Liyanage, Y. Wu, S. Tagare, and L. Zhong, "FPGA-based distributed Union-Find decoder for surface codes," *IEEE Transactions on Quantum Engineering*, 2024.
- [59] H. Bombín, C. Dawson, Y.-H. Liu, N. Nickerson, F. Pastawski, and S. Roberts, "Modular decoding: parallelizable real-time decoding for quantum computers," *arXiv preprint arXiv:2303.04846*, 2023.
- [60] L. Lao, B. van Wee, I. Ashraf, J. Van Someren, N. Khammassi, K. Bertels, and C. G. Almudever, "Mapping of lattice surgery-based quantum circuits on surface code architectures," *Quantum Science and Technology*, vol. 4, no. 1, p. 015005, 2018.
- [61] A. Molavi, A. Xu, S. Tannu, and A. Albarghouthi, "Dependency-aware compilation for surface code quantum architectures," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA1, pp. 57–84, 2025.
- [62] G. Watkins, H. M. Nguyen, K. Watkins, S. Pearce, H.-K. Lau, and A. Paler, "A high performance compiler for very large scale surface code computations," *Quantum*, vol. 8, p. 1354, 2024.
- [63] T. LeBlond, C. Dean, G. Watkins, and R. Bennink, "Realistic cost to execute practical quantum circuits using direct Clifford+ T lattice surgery compilation," *ACM Transactions on Quantum Computing*, 2023.
- [64] D. Herr, F. Nori, and S. J. Devitt, "Optimization of lattice surgery is NP-hard," *Npj quantum information*, vol. 3, no. 1, p. 35, 2017.
- [65] C. Zhu, X. Wu, J. Chen, K. He, J. Wu, X. Wang, and L. Lao, "O3LS: Optimizing lattice surgery via automatic layout searching and loose scheduling," *arXiv preprint arXiv:2604.15099*, 2026.
- [66] M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram, and A. Vassillo, "Assessing requirements to scale to practical quantum advantage," *arXiv preprint arXiv:2211.07629*, 2022.
- [67] M. Beverland, V. Kliuchnikov, and E. Schoute, "Surface code compilation via edge-disjoint paths," *PRX Quantum*, vol. 3, no. 2, p. 020342, 2022.
- [68] K. Hamada, Y. Suzuki, and Y. Tokunaga, "Efficient and high-performance routing of lattice-surgery paths on three-dimensional lattice," *arXiv preprint arXiv:2401.15829*, 2024.